

Act. 5: Técnica Voraz Huffman

Análisis de Algoritmos

Juan Ernesto López Arce Delgado



Equipo César - D01

Aurelio Rendon Viciego - ICOM
Humberto de Jesus Pena Dueñas - ICOM
Ricardo Alejandro Rosas Arreola - ICOM

Introducción

La **codificación Huffman** es un método de compresión de datos sin pérdida que utiliza un enfoque voraz para construir un árbol binario óptimo basado en la frecuencia de aparición de los caracteres en un conjunto de datos. Este algoritmo fue desarrollado por **David A. Huffman** en 1952 y se utiliza ampliamente en aplicaciones de compresión de archivos y transmisión de datos.

El poder de la Codificación Huffman reside en su naturaleza **voraz (greedy)** para construir el árbol de prefijos óptimo.

Construcción del Árbol (El Proceso Voraz)

El objetivo es crear un **árbol binario completo** (donde los caracteres son las hojas y los nodos internos representan la suma de frecuencias) que minimice la longitud total esperada del mensaje codificado. El proceso se realiza mediante una **cola de prioridad** (mínima), y sigue estos pasos:

- **Paso 1: Nodos Hoja.** Se crea un nodo hoja para cada carácter único en los datos, asignándole un peso igual a su frecuencia de aparición. Se insertan todos estos nodos en una cola de prioridad, ordenados por su peso (frecuencia) de forma ascendente.
- **Paso 2: Combinación Iterativa.** Mientras haya más de un nodo en la cola:
 - Se extraen los dos nodos con la menor frecuencia.
 - Se crea un **nuevo nodo interno** que tiene a los dos nodos extraídos como hijos.
 - El peso del nuevo nodo interno es la suma de los pesos de sus hijos.
 - Se inserta el nuevo nodo interno de vuelta en la cola de prioridad.
- **Paso 3: Raíz del Árbol.** El último nodo que queda en la cola de prioridad es la raíz del Árbol de Huffman.

Asignación de Códigos (Códigos Prefijos Óptimos)

Una vez que el árbol está construido:

- Se recorre el árbol desde la raíz hasta cada hoja (carácter).
- Se asigna un **'0'** a la rama izquierda y un **'1'** a la rama derecha.
- El código Huffman para un carácter es la secuencia de bits acumulada en el camino desde la raíz hasta su nodo hoja.

Importante: Al ser un **código prefijo**, ningún código es un prefijo de otro. Esto garantiza que la decodificación sea *única y eficiente*; al leer la secuencia de bits, en cuanto se llega a un código de hoja, se sabe inmediatamente que ese es el carácter codificado, sin necesidad de mirar bits posteriores.

Ventajas y Óptimalidad

- **Optimalidad entre Códigos Prefijos:** Huffman no solo es un buen algoritmo, es el **óptimo** para generar códigos prefijos con longitudes de bits enteras basándose en frecuencias estáticas. Esto significa que proporciona la codificación más eficiente (el menor número de bits promedio por símbolo) para una distribución de frecuencia dada.
- **Reducción de la Redundancia:** El algoritmo explota la disparidad en la frecuencia de caracteres para minimizar el número promedio de bits por carácter, acercándose al límite teórico de compresión conocido como la **entropía de Shannon** para esa fuente de datos.

Objetivo

El objetivo principal es implementar y analizar el algoritmo de Huffman, que emplea la técnica voraz para la compresión de datos, mediante la construcción de un árbol binario óptimo basado en la frecuencia de aparición de los caracteres. Además, se busca comprender a fondo el funcionamiento interno de dicho algoritmo, evaluando su eficiencia en términos de reducción del tamaño de los datos, así como su desempeño en distintos escenarios de prueba. Este análisis permitirá no solo validar la correcta implementación del algoritmo, sino también identificar posibles mejoras, aplicaciones prácticas y limitaciones dentro de procesos de compresión de información en entornos reales.

Desarrollo

Se desarrolló el programa utilizando la librería Tkinter para implementar una interfaz gráfica de usuario (GUI) para este algoritmo.

1. *Lee un archivo de texto (.txt) y calcula la frecuencia de aparición de cada carácter.*

Este fragmento se encarga de definir el objeto fundamental del algoritmo (Nodo) y la lógica para leer el archivo y calcular la frecuencia de cada carácter, que es el punto de partida de la compresión.

Compresor Huffman

Python

```
def construir_tabla_frecuencias(texto):  
    """Calcula la frecuencia de aparición de cada carácter en el texto."""  
    return Counter(texto)
```

2. *Utiliza la técnica voraz del algoritmo de Huffman para construir un árbol binario.*

Este fragmento implementa el corazón del algoritmo: la técnica voraz. Utiliza una cola de prioridad (heap) para construir el árbol binario, combinando siempre los dos nodos de menor frecuencia .

Compresor Huffman

Python

```
def construir_arbol_huffman(tabla_frecuencias):  
    monticulo = [Nodo(caracter, freq) for caracter, freq in tabla_frecuencias.items()]  
    heapq.heapify(monticulo)  
  
    while len(monticulo) > 1:  
        izq = heapq.heappop(monticulo)  
        der = heapq.heappop(monticulo)  
  
        combinado = Nodo(frecuencia=izq.frecuencia + der.frecuencia)  
        combinado.izquierda = izq  
        combinado.derecha = der  
  
        heapq.heappush(monticulo, combinado)  
  
    return monticulo[0] if monticulo else None
```

3. Genera los códigos binarios asociados a cada carácter.

Este fragmento define la función recursiva que recorre el árbol de Huffman para asignar los códigos binarios. La regla es simple: '0' si va a la izquierda, '1' si va a la derecha.

```
Compresor Huffman Python

def codificar_texto(texto, mapa_codigos):
    """Convierte el texto original a su representación binaria usando los códigos de Huffman."""
    return ''.join(mapa_codigos[caracter] for caracter in texto)

def decodificar_texto(texto_codificado, arbol):
    """Recorre el árbol de Huffman usando los bits codificados para reconstruir el texto original."""
    resultado = []
    nodo = arbol

    if arbol is None:
        return ""

    if arbol.caracter is not None:
        return arbol.caracter * len(texto_codificado)

    for bit in texto_codificado:
        nodo = nodo.izquierda if bit == '0' else nodo.derecha
        if nodo.caracter:
            resultado.append(nodo.caracter)
            nodo = arbol

    return ''.join(resultado)
```

4. Codifica y decodifica el contenido del archivo.

Las funciones **codificar_texto()** y **decodificar_texto()** implementan el uso práctico del mapa de códigos y el árbol, respectivamente.

```
Compresor Huffman Python

def generar_codigos(nodo, prefijo="", mapa_codigos=None):
    """Recorre el árbol de Huffman y asigna códigos binarios (0 a izquierda, 1 a derecha)."""
    if mapa_codigos is None:
        mapa_codigos = {}

    if nodo.caracter is not None:
        mapa_codigos[nodo.caracter] = prefijo if prefijo else "0"
    else:
        generar_codigos(nodo.izquierda, prefijo + "0", mapa_codigos)
        generar_codigos(nodo.derecha, prefijo + "1", mapa_codigos)

    return mapa_codigos
```

5. Compara el tamaño del archivo original con el tamaño del archivo codificado para evaluar la eficiencia de la compresión.

El método **codificar_y_comparar()** calcula y muestra el tamaño en bits del texto original (asumiendo 8 bits/carácter) contra el tamaño en bits de la cadena codificada de Huffman, mostrando la relación de compresión en porcentaje.

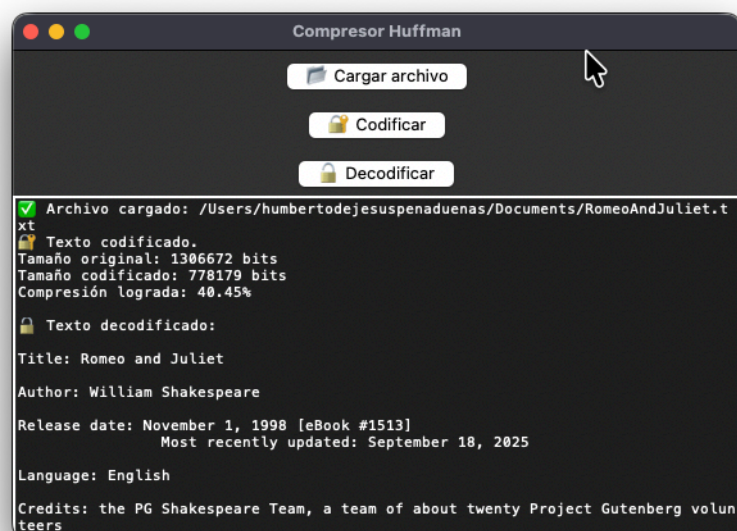
```
Compresor Huffman Python

def codificar_y_comparar(self):
    """Ejecuta los pasos de codificación y la comparación de eficiencia."""
    if not self.texto_original:
        messagebox.showerror("Error", "No se ha cargado ningún archivo.")
        return

    # Comparación
    tamaño_bytes_original = len(self.texto_original.encode('utf-8'))
    tamaño_bits_original = tamaño_bytes_original * 8
    tamaño_bits_codificado = len(self.texto_codificado)

    if tamaño_bits_original > 0:
        compresion_porcentaje = 100 - (tamaño_bits_codificado /
tamaño_bits_original * 100)
    else:
        compresion_porcentaje = 0
```

La ejecución nos muestra la **GUI** y cómo es que funciona en la práctica nuestro algoritmo de compresión:



Lo que podemos observar es una codificación o compresión del libro *Romeo and Juliet* en formato de texto (.txt), en el cual se observa una compresión lograda de hasta 40.45%. Posterior a la compresión se descomprime este mismo archivo y se muestra su contenido para poder leer/visualizarse.

Conclusiones

La prueba de codificar el texto de *Romeo and Juliet* fue un éxito rotundo, alcanzando una impresionante reducción de **más del 40%** en el tamaño del archivo. Esto confirma que el método de Huffman es extremadamente efectivo.

Al igual que en su proyecto anterior, donde el algoritmo inteligente superó al método de fuerza bruta, Huffman supera la manera común de guardar información. En lugar de usar la misma cantidad de espacio para cada letra (como hace el método estándar), Huffman analiza el texto y le da códigos cortitos a las letras que más aparecen (como los espacios o la vocal 'a'). Las letras raras obtienen códigos más largos.

Esta estrategia es lo que nos da esa gran compresión del 40%, porque aprovechamos el hecho de que el inglés (y cualquier idioma) es muy redundante. Además, la prueba de que el archivo se descomprime perfectamente demuestra la excelencia del diseño de Huffman: no se pierde ni un solo carácter, cumpliendo con el objetivo de la compresión sin pérdida.

Referencias

Huffman, D. A. (1952). A Method for the Construction of Minimum-Redundancy Codes. *Proceedings of the I.R.E.*, 40(9), 1098–1101.

<https://www.gutenberg.org/cache/epub/1513/pg1513.txt>

<https://programmerclick.com/article/8812529184/>

<https://www.geeksforgeeks.org/dsa/huffman-coding-greedy-algo-3/>

<https://revistas.unc.edu.ar/index.php/REM/article/download/17191/16809>